

Kom8ndor's Developer Guide

Last update: July 2026

Auto-generated from source analysis & Manually edited by F. Wilhelmi

Contents

1	Introduction	4
1.1	Scope of This Guide	4
2	Repository Layout	4
2.1	Source Files at a Glance	5
2.1.1	Code/COST	5
2.1.2	Code/input	5
2.1.3	Code/learning_modules/	5
2.1.4	Code/main	6
2.1.5	Code/methods	6
2.1.6	Code/scripts_multiple_executions	8
2.1.7	Code/structures	9
2.1.8	Code/config_models	10
2.1.9	Code/list_of_macros.h	10
2.1.10	Code/validate.sh	11
3	The CompC++/COST Framework	11
3.1	Key Concepts	11
3.2	Build Process	12
4	Simulation Bootstrap	12
5	Components	13
5.1	Komondor main	13
5.2	The Node Component	14
5.2.1	Inports, Outports, and Timers	14
5.2.2	Key Member Variables	15
5.3	The TrafficGenerator Component	16
5.4	The Agent Component	16
5.5	The CentralController Component	16
6	Finite-State Machine	17
6.1	Event Dispatch Architecture	18
6.2	Transmission Flow	18
6.3	Interruption Handling	19
6.4	Backoff Mechanism	19
6.5	Packet Handling	20
6.5.1	PrepareNewTransmission()	20
6.5.2	GenerateNotification()	20
6.5.3	StartTransmission()	20
6.5.4	MyTxFinished()	20

6.5.5	SendResponsePacket()	20
6.5.6	Timeout Handlers	21
7	Data Structures	21
7.1	NodeParameters	21
7.2	Wlan	21
7.3	Notification and TxInfo	22
7.4	Performance	22
7.5	Configuration and Action	22
8	Channel and Power Model	23
8.1	Path Loss and Received Power	23
8.2	SINR Computation	23
8.3	Channel Bonding	23
8.4	Power Sensing Update	23
9	Beamforming	24
9.1	Array Model	24
9.2	Algorithms	24
9.3	Integration with Transmission	24
10	Backoff models	24
11	Multi-AP Coordination (MAPC)	25
11.1	Frame Exchange Sequence	25
11.2	Multi-Group Support	26
11.3	Co-TDMA TXOP Split	26
11.4	Co-SR Power Computation	26
12	NPCA and DSO	26
12.1	DSO (Dynamic Subband Operation)	26
12.2	NPCA (Non-Primary Channel Access)	26
13	Machine Learning Subsystem	27
13.1	Agent's Flow	27
13.2	Learning Algorithms	27
13.3	Reward Types	27
13.4	External ML Server Integration	28
13.5	Central Controller	28
14	Input Configuration	28
14.1	Node CSV	28
14.2	MAPC CSV	28
14.3	Agent CSV	29
14.4	Example Run	29
15	Output	29
16	Coding Conventions	29
17	Adding a New Feature: Checklist	30
18	Common Pitfalls	30

19 Quick Reference: Key Methods	31
20 Glossary	32

1 Introduction

Kom8ndor¹ is a discrete-event IEEE 802.11bn-oriented network simulator written in C++98 on top of the CompC++/COST simulation framework. It models the full MAC/PHY protocol and includes functionalities such as channel bonding, spatial reuse, beamforming, EDCA, and Multi-AP Coordination (MAPC). It also integrates Machine Learning (ML) functionalities through a Reinforcement Learning (RL) subsystem for online parameter tuning and a socket-based integration for Python.

The codebase is approximately 112000 lines of code spread across roughly 30 header files and 5 .cc translation units. Almost all logic lives in .h files because the COST preprocessor inlines component method bodies.

1.1 Scope of This Guide

This guide is intended to introduce the internal behavior of Kom8ndor to any interested researcher/practitioner who would like to extend/modify the source code. In particular, this document provides:

- A map of the directory structure and file responsibilities.
- An explanation of the CompC++/COST execution model.
- A walk-through of the node finite-state machine (FSM).
- Descriptions of every major subsystem: backoff, channel access, power/SINR, beamforming, MAPC, NPCA/DSO, EDCA, and ML.
- Input/output CSV formats and how to create new scenarios.
- Build instructions.

2 Repository Layout

Komondor/	
Apps/	Post-processing & plotting utilities
Code/	
COST/	CompC++ simulation framework (Linux ELF)
input/	Input scenarios (CSV files)
learning_modules/	ML/RL components
main/	Main files & components and Makefile
methods/	Implementation logic headers
channel/	Power, SINR, beamforming
frames/	Notifications, aggregation logic
mac/	Backoff, channel access, NAV, losses
mapc/	Multi-AP Coordination methods
node/	Node FSM helpers, packet, timeout, backoff
utils/	Input loader, output writer
phy/	Modulations, beamforming, channel bonding
output/	Placeholder for simulation outputs
scripts_multiple_executions	Examples of scripts to run multiple scenarios
structures/	Data structures (NodeParameters, Wlan...)
config_models	File with global models
list_of_macros	File with macros
validate	File that runs regression tests
Documentation/	Docs & tutorials

Listing 1: Top-level directory tree.

¹This is the name we gave to the Wi-Fi 8 extension of Komondor.

2.1 Source Files at a Glance

We next list every source file under `Code/` with its approximate role. More detailed descriptions of each block are provided throughout this document.

2.1.1 Code/COST

The COST folder is a third-party discrete-event simulation framework from Rensselaer Polytechnic Institute (authors: Gilbert Chen, Boleslaw Szymanski, Mark Lisee; circa 2003–2007). COST stands for Component-Oriented Simulation Toolkit, layered here with the SENSE (Simulator for wirEless Network Simulation Experiments) library on top of it.

2.1.2 Code/input

Includes input CSV files to define simulation scenarios. It contains two baseline folders:

- `Code/input/examples`: Contains examples for different features (bandwidth adaptation, basic example, DSO and NPCA, EDCA, external ML model...).
- `Code/input/validation`: Contains the files used by the regression tests (see `validate.sh`).

2.1.3 Code/learning_modules/

- `action_space.h`: Action encoding/decoding.
- `agent_methods.h`: Agent helper functions, including performance-metric reset and data-validity checks.
- `external_model_client.h`: Unix-socket client for external Python ML servers.
- `learning_algorithm.h`: Algorithm dispatcher (MAB, RTOT², external model).
- `ml_model.h`: Legacy ML model (used by `CentralController`).
- `pre_processor.h`: Thin coordinator over reward/action/algorithm.
- `reward_function.h`: Reward computation (throughput-based, fairness...).
- `Code/learning_modules/network_optimization_methods/`
 - `centralized_action_banning.h`: Used to ban low-performing actions across all agents under central-controller coordination, with rollback support.
 - `multi_armed_bandits.h`: Implements ϵ -greedy, UCB1, and Thompson Sampling strategies.³
 - `rtot_algorithm.h`: `RtotAlgorithm` class implementing the Recommended TX Output Threshold algorithm for adaptive OBSS-PD tuning.
- `Code/learning_modules/python_servers/`
 - `ml_server_passthrough.py`: Smoke-test server that echoes the received arm index back unchanged; useful for verifying socket connectivity.
 - `ml_server_random.py`: Baseline server that selects a random arm each step; reads the number of arms from the feature vector sent by the simulation.
 - `ml_server_pytorch.py`: PyTorch inference / online-RL server; loads a TorchScript model and returns the action predicted by a forward pass.

²Ropitault, T. (2018, January). Evaluation of RTOT algorithm: A first implementation of OBSS_PD-based SR method for IEEE 802.11 ax. In 2018 15th IEEE Annual Consumer Communications & Networking Conference (CCNC) (pp. 1-7). IEEE.

³Lattimore, T., & Szepesvári, C. (2020). Bandit algorithms. Cambridge University Press.

2.1.4 Code/main

- **agent.h**: ML agent component, with interactions with nodes, other agents, and the central controller.
- **build_local**: Script that builds the code from the pre-compiled cxx COST engine (platform-dependent). The Makefile (added later and the preferred way to build now) does the same steps but also adds dependency tracking (it watches all .cc and .h files and rebuilds only when something has changed).
- **central_controller.h**: Centralized multi-WLAN coordination that handles agents associated with it.
- **komondor_main.cc**: Entry point, including CLI parsing, component wiring, and simulation loop.
- **Makefile**: Defines build rules to compile the source code (`g++ -std=c++98`).
- **node.h**: Monolithic node component, including all the interactions that occur during the simulation.
- **traffic_generator.h**: Module that generates traffic for nodes. Includes traffic models such as *full-buffer*, *Poisson*, and *deterministic*.

2.1.5 Code/methods

- *Code/methods/frames/*
 - **notification_methods.h**: Builds the TxInfo struct embedded in every Notification (GenerateTxInfo); called by GenerateNotification() inside node.h.
 - **packet_aggregation_methods.h**: Computes the maximum A-MPDU aggregation level (FindMaximumPacketsAggregated) given the current MCS, channel bandwidth, and remaining TXOP budget.
- *Code/methods/mac/*
 - *Code/methods/mac/channel_access/*
 - * **backoff_methods.h**: Binary exponential backoff and EDCA backoff dispatcher; entry point for all backoff-type computations.
 - * **channel_access_methods.h**: Channel access policy abstraction (ChannelAccessPolicy strategy struct); registers the CCA-check and channel-bonding selection logic used when a backoff expires.
 - * **dcf_methods.h**: DCF random-slot backoff (uniform draw in [0, CW)).
 - * **deterministic_backoff_methods.h**: Deterministic backoff schedule (Qualcomm variant⁴).
 - * **eca_methods.h**: Enhanced Channel Access (ECA) backoff variant.⁵
 - * **edca_methods.h**: EDCA per-access-category binary exponential backoff with AIFSN computation.

⁴Kosek-Szott, K., Szott, S., Valvo, A. L., & Tinnirello, I. (2022, October). Db-lbt: Deterministic backoff with listen before talk for wi-fi/nr-u coexistence in shared bands. In 2022 30th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS) (pp. 168-175). IEEE.

⁵Sanabria-Russo, L., Barcelo, J., Bellalta, B., & Gringoli, F. (2016). A high efficiency MAC protocol for WLANs: Providing fairness in dense scenarios. IEEE/ACM Transactions on Networking, 25(1), 492-505.

- * `tokenized_backoff_methods.h`: Token-based channel access backoff.⁶
 - `nack_methods.h`: Generates and processes logical NACKs for collision and decoding-failure events (`GenerateLogicalNack`, `ProcessNack`).
 - `nav_methods.h`: Computes NAV duration values (`ComputeNavTime`) for RTS, CTS, DATA, ACK, ICF, and TF frames, so that neighbouring nodes set their virtual carrier-sense timer correctly.
 - `packet_loss_methods.h`: Packet loss decision logic (`HandlePacketLoss`, `CanDecodePacket`, `IsPacketLost`); applies the SINR threshold, capture-effect model, and constant PER override.
 - `spatial_reuse_methods.h`: BSS-colour-based SR helpers: packet-origin classification (`CheckPacketOrigin`), OBSS-PD sensitivity selection (`GetSensitivitySpatialReuse`), SR opportunity identification (`IdentifySpatialReuseOpportunity`), and TX-power restriction (`ApplyTxPowerRestriction`).
- *Code/methods/mapc/*
 - `mapc_methods.h`: MAPC coordinator helpers for peer-AP selection logic (`selectMapcPeer`, used for round-robin TXOP allocation) and Co-SR transmit-power calculation (used in the coordinated power adaptation path).
- *Code/methods/node/*
 - `node_backoff_methods.h`: DIFS/EIFS handling, EndBackoff dispatch.
 - `node_fsm_methods.h`: Setup, Start, Stop, InitializeVariables, RestartNode.
 - `node_packet_methods.h`: PrepareNewTransmission, GenerateNotification, MyTxFinished.
 - `node_timeout_methods.h`: ACK/CTS/DATA/NPCA timeout handlers.
- *Code/methods/phy/*
 - `beamforming_methods.h`: ULA beam-weight computation for standalone and coordinated beamforming, including projection null-steerer (Gram–Schmidt, `ComputeBeamWeights`) and Zero-Forcing precoding (`ComputeZFBeamWeights`), plus `ComputeRxBeamGain` to apply the resulting gain at the receiver.
 - `channel_bonding_methods.h`: Channel-bonding selection logic for IEEE 802.11ax and beyond. The main entry point `GetTxChannelsByChannelBondingCCA11ax` evaluates per-subchannel CCA thresholds (primary at -82 dBm, secondaries at -72 dBm) and returns the widest feasible contiguous power-of-two bandwidth, up to 320 MHz, for all supported bonding models.
 - `frame_duration_methods.h`: Frame-duration calculations for all IEEE 802.11ax MAC frame types: `ComputeTxTime` (generic bits/rate formula), `ComputeRtsTxTime80211ax`, `ComputeCtsTxTime80211ax`, and `ComputeDataTxTime80211ax` (accounts for A-MPDU aggregation level and packet length). Also defines the `Exponential` random-variate helper used for stochastic transmission-time modelling.
 - `mcs_methods.h`: MCS selection and link-quality helpers. `SelectMCSResponse` maps a received power level to the highest feasible MCS index using RSSI thresholds that scale with channel width (+3 dB per doubling of bandwidth). `ComputeEbToNoise` converts SINR, bit rate, bandwidth, and modulation type into an E_b/N_0 value for PER evaluation.

⁶Wilhelmi, F., Galati-Giordano, L., & Fontanesi, G. (2025, March). “It’s Your Turn”: A Novel Channel Contention Mechanism for Improving Wi-Fi’s Reliability. In 2025 IEEE Wireless Communications and Networking Conference (WCNC) (pp. 1-6). IEEE.

- `power_channel_methods.h`: Path-loss computation, per-node received-power tracking (`UpdatePowerSensedPerNode`, with beamforming gain applied to DATA frames only), SINR calculation, and the `ConvertPower` utility for unit conversions between pW, mW, and dBm.
- *Code/methods/utis/*
 - *Code/methods/utis/input_methods/*
 - * `input_loader.h`: CSV parsing for all input files. Reads `config_models` and performs a two-pass parse of the node CSV (first pass identifies WLANs, second pass instantiates nodes and STAs); further functions load MAPC group files and agent CSV files.
 - * `input_validator.h`: Post-load sanity checks on the assembled node container (`ValidateInput`): verifies TX power and sensitivity are within admissible ranges, checks that channel assignments are consistent, and detects duplicate node IDs or positions.
 - `auxiliary_methods.h`: General-purpose helpers used across the codebase: `ToString` (generic value-to-string conversion), array utilities (e.g., `PickRandomElementFromArray`), and numeric helpers (`RandomDouble`, `TruncateDouble`, `RoundToDigits`).
 - `logging_methods.h`: Low-level log-writing helpers shared by all components. Formats integer and floating-point arrays into the node log file or console.
 - `output_generation_methods.h`: End-of-simulation statistics and formatted output. Aggregates per-node `Performance` objects into a `SimulationStats` summary and dispatches to one of several scenario-specific formatters (toy two-WLAN scenario, large node-density scenario, central-WLAN detailed counters, Bianchi multi-WLAN analysis, etc.) selected by the `simulation_ix_output_script` field in `config_models`.
 - `print_and_write_methods.h`: Kom8ndor-level diagnostic printers called at simulation start and end (e.g., `PrintSystemInfo/WriteSystemInfo`). These produce the human-readable header block visible at the top of every log file.

2.1.6 Code/scripts_multiple_executions

Contains three Bash helper scripts for running batches of simulations. All scripts trigger a `make` build before executing and append every result to a single output file. They are run from the `Code/scripts_multiple_executions/` directory on Linux, and their key parameters (input folder, number of seeds, agents file) can be overridden via environment variables without editing the scripts.

- `multiple_inputs_script.sh`: Runs `komondor_main` once per `input_nodes*.csv` file found in a given input folder, using a fixed seed. Useful for running simulation campaigns.
- `multiple_inputs_script_several_seeds.sh`: Extends the above by repeating each scenario file across N different random seeds (default 5), allowing statistical averaging of results.
- `multiple_inputs_script_agents.sh`: Runs `komondor_main` with a shared agents CSV file paired against every node file in a given folder. Intended for ML-agent experiments where the same learning configuration is evaluated across multiple topologies.

2.1.7 Code/structures

- **action.h**: One discrete arm in the learning action space, storing the associated channel, CCA threshold, TX power, and maximum bandwidth, together with reward statistics and play counters tracked since the last controller request.
- **agent_capabilities.h**: Advertises to the central controller which action dimensions (channel, power, sensitivity, bandwidth) a given agent is allowed to tune.
- **channel_access_state.h**: Groups the live backoff countdown, current contention-window stage and bounds, deterministic-backoff phase flag, and ECA/repeat-BO state that would otherwise be scattered as private variables in **node.h**.
- **controller_report.h**: The shared state held by the central controller, including per-agent last configuration and performance snapshots, cluster membership arrays, and per-agent action-statistics matrices used for centralized action banning.
- **FIFO.h**: A simple packet queue wrapping an array of **Notification** objects, with methods to enqueue (**PutPacket**, **PutPacketFront**, **PutPacketIn**), dequeue (**DelFirstPacket**, **DeletePacketIn**), and inspect (**GetFirstPacket**, **GetPacketAt**, **QueueSize**).
- **logger.h**: A file pointer paired with a write-enable flag, passed into every method that produces per-node log output.
- **logical_nack.h**: Carries the packet type, packet ID, source and destination node IDs, and loss-reason code exchanged when a transmission fails.
- **modulations.h**: Look-up tables for the MCS entries (modulation bits and coding rates), plus the helper **GetNumberSubcarriers()** that maps a channel count to the corresponding OFDM subcarrier count (20 MHz through 320 MHz).
- **node_configuration.h**: The runtime tunable state of a node: primary channel, packet-detection threshold, TX power, maximum bandwidth, and spatial-reuse parameters. This is the type exchanged between nodes and their ML agents.
- **node_parameters.h**: Static per-node configuration loaded once from the node CSV (identity, position, channel range, channel-bonding model, default TX power, CCA threshold, traffic model, protocol variant, etc.).
- **node_statistics.h**: All runtime counters and measurements for a node, organized into categories: packet/frame counters, throughput/delay/utilization, backoff history, channel-idle tracking, NAV tracking, MAPC/TXOP-sharing statistics, rolling-window per-period stats, and dynamically allocated per-channel and per-STA arrays.
- **notification.h**: The message broadcast between nodes on every transmission event, carrying source/destination IDs, packet type, channel range, TX power, NAV time, MCS, beamforming parameters, MAPC allocation fields, spatial-reuse fields, and preamble-puncturing mask.
- **packet_exchange_sequence.h**: Defines the ordered list of MAC frame types for each supported exchange mode, with pre-built static instances: **IEEE_802_11_NO_RTS_CTS**, **IEEE_802_11_RTS_CTS**, **IEEE_802_11_COTDMA**, **IEEE_802_11_COBF_COSR**, **IEEE_802_11_DS0**, and **IEEE_802_11_NPCA**.
- **performance.h**: Per-measurement-window snapshot reported from a node to its agent: throughput, maximum-bound throughput, packets sent/lost/ACKed, RTS/CTS counters, delay statistics, channel utilization, per-WLAN RSSI list, and received-power array.

- **sim_configuration.h**: Aggregates all CLI and **config_models** inputs into a single object passed through the simulation: logging flags, simulation time and seed, feature-enable flags (agents, MAPC, beamforming, DSO, NPCA, ...), physical-layer model selections, and agent action-space definitions.
- **simulation_stats.h**: Aggregated statistics accumulated across all nodes for the full simulation run, replacing former module-scope global variables.
- **spatial_reuse_state.h**: Groups the spatial-reuse runtime state (SR enable flag, TXOP-SR identification flag, current OBSS-PD threshold, TX-power restriction, Co-SR active flag, and related fields) that would otherwise be private variables in **node.h**.
- **wlan.h**: WLAN-level configuration: AP and STA node IDs, channel-bonding policy, MAPC group arrays (up to **MAX_MAPC_GROUPS_PER_WLAN**), beamforming parameters, DSO/NPCA enable flags, and EDCA per-AC parameters.

2.1.8 Code/config_models

A plain-text key-value configuration file (no extension) read by the simulator at start-up to set global physical-layer and MAC model choices independently of any scenario CSV. The fields it controls are:

- **adjacent_channel_model**: Selects the adjacent-channel interference model.
- **collisions_model**: Selects how simultaneous-transmission collisions are resolved.
- **path_loss_model**: Selects the propagation model applied to all node pairs (e.g., free-space, Okumura-Hata urban, IEEE 802.11ax indoor).
- **simulation_ix_output_script**: Integer index appended to batch-script output filenames for book-keeping.

2.1.9 Code/list_of_macros.h

The single header included by virtually every other file. It is the authoritative source of all named integer constants used across the simulator and is organized into the following groups:

- **Boolean**: TRUE/FALSE.
- **Node FSM states**: core 802.11 states (**STATE_SENSING**, **STATE_TX_DATA**, **STATE_NAV**, etc.), MAPC coordination states (**STATE_TX_ICF**, **STATE_RX_ICR**, **STATE_TX_TF**, ...), DSO states (**STATE_TX_DSO_ICF**, ...), and NPCA states (**STATE_TX_NPCA_ICF**, ...).
- **Node types**: **NODE_TYPE_AP** / **NODE_TYPE_STA**.
- **Packet types**: **PACKET_TYPE_DATA**, **ACK**, **RTS**, **CTS**, **MCS_REQUEST**, **MCS_RESPONSE**, and all MAPC (**ICF**, **ICR**, **TF**, **MU_RTS_TXS**), DSO, and NPCA frame types.
- **Logical NACK reasons**: enumeration of collision and loss causes.
- **Path-loss models**: free-space (**PATH_LOSS_LFS**), Okumura-Hata (**PATH_LOSS_OKUMURA_HATA**), indoor (**PATH_LOSS_INDOOR**), and others.
- **Channel bonding models**: CCA-policy flags (**CHANNEL_AGGREGATION_CCA_SAME**, etc.).
- **Traffic and QoS**: traffic-model identifiers and access-category (AC) constants.

- **EDCA parameters:** per-AC contention-window bounds (`CW_MIN_AC_VO` ... `CW_MAX_AC_BK`), AIFSN values (`AIFSN_VO` ... `AIFSN_BK`), and TXOP limits per AC per IEEE 802.11-2020 Table 9-155.
- **Timing and power:** slot time, SIFS, DIFS, power-unit conversion factors, CCA threshold defaults.
- **Miscellaneous:** log-level constants, progress-bar flag, channel-boundary sentinels, and special destination IDs (e.g., `NODE_ID_MAPC_BROADCAST`).

2.1.10 Code/validate.sh

A Bash regression-test script run from the `Code/` directory on Linux (`bash validate.sh`). It executes the compiled binary against the scenarios in `Code/input/validation/` and compares key output metrics against a pre-refactor baseline, reporting `[PASS]`, `[FAIL]`, or `[WARN]` for each check. The script is organized into four parts:

- **Part 1 - Basic scenarios:** scenarios 1a and 1b, with and without A-MPDU aggregation; checks WLAN-A throughput within ± 1 Mbps of the baseline.
- **Part 2 - Complex scenarios:** scenarios 2a–2d, with and without aggregation, involving up to three overlapping WLANs; checks per-WLAN throughput (WLAN-A, B, C) within ± 1 Mbps.
- **Part 3 - Channel-bonding scenarios:** 20, 40, 80, and 160 MHz configurations; checks WLAN-A received RSSI within ± 0.1 dBm.
- **Part 4 - Spatial-reuse scenarios:** SR enabled (4a) and disabled (4b); checks WLAN-A and WLAN-B throughput within ± 1 Mbps.

A summary line is printed at the end with the total number of passed, failed, and warned checks. The script exits with a non-zero status if any check fails (`set -euo pipefail`).

3 The CompC++/COST Framework

Komondor is built on CompC++ (Component-oriented C++), a discrete-event simulation framework. The COST preprocessor (`Code/COST/cxx`, a Linux ELF binary) transforms `.cc` files containing `component` declarations into standard C++ that links against the COST runtime.

3.1 Key Concepts

- **Component:** A self-contained simulation entity declared with the `component` keyword. Komondor defines three components: `Node`, `Agent`, and `CentralController`.
- **Inport:** An event-receiving method. When another component sends a message to this port, the method is invoked synchronously at the scheduled simulation time.
- **Outport:** An event-sending port. Calling an outport broadcasts a message to all connected inports. Connections are established in `komondor_main.cc`.
- **Timer (Trigger):** A self-scheduled event. A component arms a timer with a future simulation time; when that time is reached, the associated handler fires.
- **SimTime():** Returns the current simulation time (a `double` in seconds).
- **TypeII base class:** All Komondor components inherit from `TypeII`, which provides the inport/outport/timer infrastructure.

3.2 Build Process

To build the code, run the following commands from Komondor main directory:

```
cd Code/main
make clean      # remove komondor_main and komondor_main.cxx
make            # full build
```

Listing 2: Build steps with Makefile (Linux only)

The Makefile performs two steps automatically:

1. **Code generation** — if `komondor_main.cc`, the COST wrapper (`../COST/cxx`), or any `.cc/.h` file in `Code/main/` has changed since the last build, the COST preprocessor transforms `komondor_main.cc` into standard C++ (`komondor_main.cxx`).
2. **Compilation** — `g++ -Wall -g -std=c++98` compiles `komondor_main.cxx` into the `komondor_main` binary (or `komondor_main.exe` on Windows, detected automatically via the `OS/COMSPEC` environment variables).

Dependency tracking covers all `.cc` and `.h` files in `Code/main/` via a wildcard rule, so any header change triggers a full rebuild. The `clean` target removes both the binary and the intermediate `.cxx` file.

Alternatively, the build process can be done using the old `build_local` file from `Code/main`, which does the following:

```
# Step 1: COST preprocessor transforms .cc -> .cxx
../COST/cxx komondor_main.cc
# Step 2: Compile with g++ (C++98 strict)
g++ -Wall -Werror -g -o komondor_main komondor_main.cxx
```

Listing 3: Build steps with `build_local` (Linux only)

Important: The COST preprocessor is a Linux ELF binary. The project cannot be compiled on Windows natively.

4 Simulation Bootstrap

The entry point is `Code/main/komondor_main.cc`. The simulation proceeds through the following phases:

1. **CLI Parsing:** Reads command-line arguments: `--nodes`, `--agents`, `--time`, `--seed`, `--mapc`, etc.
2. **Node Loading:** Parses the node CSV file to create a `NodeParameters` array.
3. **Input checker:** Checks that the parameters from the nodes file are correct (e.g., the primary channel is within the indicated channels, two different nodes are not in the same exact position).
4. **WLAN Construction:** Groups nodes into `Wlan` structures by matching `wlan_code` strings informed in the nodes CSV.
5. **Component Instantiation:** Creates arrays of `Node`, `Agent`, and `CentralController` components.
6. **Wiring:** Connects outputs to inputs.
 - Every `Node.outportSelfStartTX` → every other `Node.InportSomeNodeStartTX`
 - Every `Node.outportSelfFinishTX` → every other `Node.InportSomeNodeFinishTX`
 - `Node.outportRequestToAgent` ↔ `Agent.InportReceivingFromNode`
 - `Agent` ↔ `CentralController`

7. **Power Matrix:** Pre-computes the received-power matrix `received_power_array[N][N]` between all node pairs, plus position arrays `all_node_x/y/z[N]` for beamforming.
8. **Simulation Run:** `Kosmos.Start()` launches the COST event loop until `simulation_time` is reached.
9. **Output:** Computes and writes per-node and per-WLAN performance statistics.

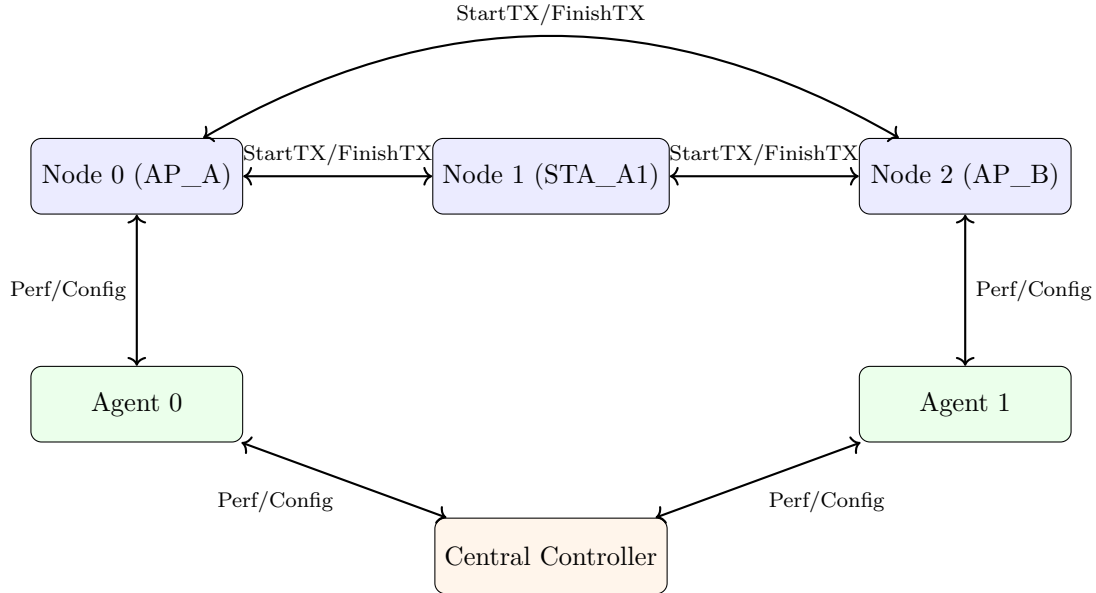


Figure 1: Component wiring for a two-BSS scenario with ML agents. Depending on the input, some nodes might not have agents (no learning) and some agents might not be attached to the controller (independent learning).

5 Components

5.1 Komondor main

`komondor_main.cc` defines the `Komondor` compound component, which inherits from `CostSimEng` and serves as the root of the simulation. It is instantiated once by `main()` and owns all other component arrays. `komondor_main` extends `COST`'s methods as follows:

- `Setup()` reads `config_models`, parses the node CSV (`GenerateNodesByReadingInputFile`), optionally parses the MAPC CSV (`GenerateMapcConfByReadingInputFile`) and the agents CSV (`GenerateAgents`, `GenerateCentralController`), pre-computes the received-power matrix, and wires all `COST` ports.
- `Start()` launches the `COST` event loop, which triggers the `Start()` from every instantiated component (nodes, agents, controller).
- `Stop()` collects per-node `Performance` objects, calls `ComputeSimulationStatistics`, and writes all output files.

To handle the different instantiated components, `Komondor` holds three `COST` component arrays (containers) as public members so that port-wiring code can access them:

- `node_container[]`: one `Node` per line in the node CSV.
- `traffic_generator_container[]`: one `TrafficGenerator` per node.

- `agent_container[]` / `central_controller[]`: instantiated only when `--agents` is supplied.

When it comes to port wiring—i.e., establishing connections between components, so that they can communicate—, `Setup()` establishes the following connection groups:

- *Traffic (one-to-one)*: A traffic generator component is connected to a node:
`traffic_generator_container[n].outportNewPacketGenerated`
 $\rightarrow$ `node_container[n].InportNewPacketGenerated`.
- *Wireless medium (all-to-all)*: every node's `outportSelfStartTX`, `outportSelfFinishTX`, and `outportSendLogicalNack` are connected to the corresponding inports of every other node.
- *Node (AP) $\leftrightarrow$ Node (STA)*: Intra-WLAN MCS request/response and WLAN-configuration update ports are connected between the AP and its associated STAs; spatial-reuse configuration ports are connected within each WLAN.
- *Node $\leftrightarrow$ Agent*: When instantiated, agents are connected to nodes to enable the exchange of performance information and commands. `outportRequestInformationToAp` / `outportAnswerToAgent` / `outportSendConfigurationToAp` matched by `wlan_code`.
- *Agent $\leftrightarrow$ CentralController*: Similarly, agents can be connected to the central controller through `outportAnswerToController` and `outportSendCommandToAgent` outports.

5.2 The Node Component

`Node` (`Code/main/node.h`) is the central component. Each `Node` instance models a single AP (0) or STA (1), indicated as `node_type` from the input nodes CSV and implementing the full 802.11 MAC/PHY state machine.

5.2.1 Inports, Outports, and Timers

Table 1: Node interface

Port / Timer	Description
<i>Inports</i>	
<code>InportSomeNodeStartTX</code>	Another node began transmitting
<code>InportSomeNodeFinishTX</code>	Another node finished transmitting
<code>InportNackReceived</code>	Logical NACK from collision
<code>InportMCSRequestReceived</code>	MCS selection request
<code>InportMCSResponseReceived</code>	MCS selection response
<code>InportNewPacketGenerated</code>	Traffic generator callback
<code>InportReceivingFromAgent</code>	ML agent sends new configuration
<i>Outports</i>	
<code>outportSelfStartTX</code>	Broadcast “I started transmitting”
<code>outportSelfFinishTX</code>	Broadcast “I finished transmitting”
<code>outportSendLogicalNack</code>	Send collision NACK
<code>outportRequestToAgent</code>	Request ML decision from agent
<code>outportAnswerToController</code>	Respond to central controller
<i>Timers</i>	
<code>trigger_start_backoff</code>	DIFS/AIFS inter-frame space

Port / Timer	Description
trigger_end_backoff	Backoff countdown expiration
trigger_SIFS	SIFS inter-frame space
trigger_toFinishTX	End of current transmission
trigger_ACK_timeout	ACK timeout (retransmission trigger)
trigger_CTS_timeout	CTS timeout
trigger_DATA_timeout	DATA timeout
trigger_NAV_timeout	NAV expiry
trigger_inter_bss_NAV_timeout	Inter-BSS NAV (SR only)
trigger_preoccupancy	1 ps delay before TX occupancy
trigger_restart_sta	STA recovery delay
trigger_wait_collisions	Backoff collision detection
trigger_dso_icr_timeout	DSO ICR wait timer
trigger_npca_switch	NPCA radio tuning delay
trigger_npca_backoff	NPCA EDCA backoff
trigger_npca_icr_timeout	NPCA ICR safety timeout
trigger_npca_timer	NPCA switch-back to primary
trigger_rho_measurement	Periodic channel utilization sampling
txop_sr_end	Spatial reuse TXOP limit

5.2.2 Key Member Variables

```

int node_state; // Current FSM state
NodeParameters node_params; // Identity from CSV
Wlan wlan; // WLAN membership

// Channel
int primary_channel, current_left_channel, current_right_channel;
double power_received_per_node[];

// Backoff (ChannelAccessState struct)
int current_CW;
double remaining_backoff;
int num_bo_interruptions;

// Transmission
Notification current_notification, rts_notification, cts_notification;

// MAPC
int coordinator_ap_id, mapc_active_group_idx;
int mapc_coordinator_group_rr;
double mapc_txop_data_budget, mapc_txop_per_ap_data_duration;

// Spatial Reuse (SpatialReuseState struct)
struct { int spatial_reuse_enabled, txop_sr_identified,
        mapc_cosr_active;
        double current_tx_power_sr, non_srg_obss_pd; } sr_state;

// Performance
double throughput, data_packets_sent, data_packets_lost;

```

Listing 4: Selected node member variables (simplified)

5.3 The TrafficGenerator Component

TrafficGenerator (`Code/main/traffic_generator.h`) is a lightweight **TypeII** component, one instance per node. It is active only for AP-type nodes (downlink transmissions only), while STAs do not generate traffic independently. A single timer, `trigger_new_packet_generated`, drives the component. When it fires, `GenerateTraffic()` is called: it computes the time until the next packet arrival according to the configured traffic model and re-arms the timer, then fires `outportNewPacketGenerated()` to notify the associated **Node**. The supported models are:

- **TRAFFIC_POISSON**: exponentially distributed inter-arrival times with mean $1/\lambda$ seconds (`Exponential(1/traffic_load)`).
- **TRAFFIC_DETERMINISTIC**: fixed inter-arrival time.
- **TRAFFIC_FULL_BUFFER**: implemented as saturated Poisson ($\lambda = 10^3$ packets/s).
- **TRAFFIC_FULL_BUFFER_NO_DIFFERENTIATION**: no timer is armed; the node manages its own buffer, which is always virtually full (data packets are generated at the time of initiating a transmission).

5.4 The Agent Component

Agent (`Code/main/agent.h`) is a **TypeII** component that implements the per-WLAN ML pipeline described in Section 13. One agent is created per WLAN when a `--agents` CSV is supplied.

The main ports of an agent are as follows:

- *Inport* `InportReceivingInformationFromAp` — receives a **Configuration** and **Performance** snapshot from the AP each time the measurement timer fires.
- *Inport* `InportReceiveCommandFromController` — receives a command ID, a new configuration, a shared performance value, and an override reward type from the **CentralController** (centralized mode only).
- *Outport* `outportRequestInformationToAp` — periodic pull request to the AP.
- *Outport* `outportSendConfigurationToAp` — pushes the new **Configuration** computed by the learning algorithm.
- *Outport* `outportAnswerToController` — forwards the agent's current configuration and performance to the **CentralController**.

Regarding agent timers, `trigger_request_information_to_ap` fires at regular intervals of `time_between_requests` seconds, triggering `RequestInformationToAp()`, which in turn causes the AP to report its **Performance** back via the inport above.

5.5 The CentralController Component

CentralController (`Code/main/central_controller.h`) is an optional **TypeII** component, instantiated at most once per simulation. It acts as the Collector and Distributor in the centralized ML pipeline: it gathers performance reports from all associated agents, runs a global optimization step, and broadcasts updated configurations back. The main ports of a central controller to interact with agents are:

- *Inport* `InportReceivingInformationFromAgent`: receives an agent ID, the agent's current **Configuration** and **Performance**, and the selected action index.

- *Outport* `outportSendCommandToAgent`: sends a command ID and a new `Configuration` to a specific agent.

Regarding timers, the central controller implements the following ones:

- `trigger_request_information_to_agents`: periodically fires `RequestInformationToAgents()`, pulling the latest state from every connected agent.
- `trigger_apply_ml_method`: fires `ApplyMLMethod()`, which invokes the `MLModel` (wrapping `CentralizedActionBanning` or a bandit) on the aggregated `ControllerReport` and dispatches updated configurations.

Additional responsibilities include the `GenerateClusters()` / `UpdatePerformancePerCluster()` methods, which optionally group WLANs by spatial proximity or interference level, enabling decision-making on a per-cluster basis. `UpdateControllerReport()` maintains the `ControllerReport` struct (per-agent action statistics, last configuration, last performance) that serves as input to every ML step.

6 Finite-State Machine

The node FSM is the heart of the simulator. Table 2 lists every state.

Table 2: Node FSM states

Constant	Value	Description
<i>Core 802.11 states</i>		
<code>STATE_SENSING</code>	0	Idle, sensing medium / backoff countdown
<code>STATE_TX_DATA</code>	1	Transmitting DATA frame
<code>STATE_RX_DATA</code>	2	Receiving DATA
<code>STATE_WAIT_ACK</code>	3	Waiting for ACK after DATA
<code>STATE_TX_ACK</code>	4	Transmitting ACK
<code>STATE_RX_ACK</code>	5	Receiving ACK
<code>STATE_TX_RTS</code>	6	Transmitting RTS
<code>STATE_TX_CTS</code>	7	Transmitting CTS
<code>STATE_RX_RTS</code>	8	Receiving RTS
<code>STATE_RX_CTS</code>	9	Receiving CTS
<code>STATE_WAIT_CTS</code>	10	Waiting for CTS after RTS
<code>STATE_WAIT_DATA</code>	11	Waiting for DATA after CTS
<code>STATE_NAV</code>	12	Virtual carrier sense (NAV)
<i>MAPC states</i>		
<code>STATE_TX_ICF</code>	14	Transmitting ICF (coordinator)
<code>STATE_WAIT_ICR</code>	15	Waiting for ICR responses
<code>STATE_TX_TF</code>	16	Transmitting Trigger Frame
<code>STATE_RX_ICF</code>	17	Receiving ICF (coordinated AP)
<code>STATE_RX_ICR</code>	18	Receiving ICR
<code>STATE_TX_ICR</code>	19	Transmitting ICR
<code>STATE_WAIT_MU_RTS</code>	20	Waiting for MU-RTS
<code>STATE_RX_MU_RTS</code>	21	Receiving MU-RTS
<code>STATE_WAIT_TF</code>	22	Waiting for TF
<code>STATE_RX_TF</code>	23	Receiving TF
<code>STATE_TX_ACK_TF</code>	24	Transmitting ACK TF

Constant	Value	Description
STATE_WAIT_ACK_TF	25	Waiting for ACK TF
<i>DSO states</i>		
STATE_TX_DSO_ICF	26	DSO ICF on full BSS bandwidth
STATE_RX_DSO_ICR	27	DSO ICR on secondary subband
STATE_TX_DATA_DSO	28	DATA on DSO subband
STATE_WAIT_ACK_DSO	29	Waiting for ACK after DSO DATA
<i>NPCA states</i>		
STATE_TX_NPCA_ICF	30	NPCA ICF on secondary channels
STATE_RX_NPCA_ICR	31	AP waiting for NPCA ICR
STATE_TX_DATA_NPCA	32	DATA on NPCA secondary channels
STATE_WAIT_ACK_NPCA	33	Waiting for ACK after NPCA DATA
STATE_TX_NPCA_ICR	34	STA transmitting NPCA ICR

6.1 Event Dispatch Architecture

The two main input handlers—`InportSomeNodeStartTX` and `InportSomeNodeFinishTX`—dispatch to state-specific handler methods:

```
void Node::InportSomeNodeStartTX(Notification &notification) {
    switch (node_state) {
        case STATE_SENSING:    HandleStartTX_StateSensing(notification)
                                ; break;
        case STATE_NAV:        HandleStartTX_StateNav(notification);
                                break;
        case STATE_TX_DATA:    HandleStartTX_StateTxData(notification);
                                break;
        case STATE_RX_DATA:    HandleStartTX_StateRxData(notification);
                                break;
        case STATE_WAIT_ACK:   HandleStartTX_StateWaitAck(notification)
                                ; break;
        case STATE_WAIT_CTS:   HandleStartTX_StateWaitCts(notification)
                                ; break;
        case STATE_WAIT_DATA:  HandleStartTX_StateWaitData(notification)
                                ; break;
        case STATE_WAIT_ICR:   HandleStartTX_StateWaitIcr(notification)
                                ; break;
        // ... MAPC/DSO/NPCA states
    }
}
```

Listing 5: StartTX dispatch (simplified)

Similarly, `InportSomeNodeFinishTX` dispatches to:

- `HandleFinishTX_StateSensing` — frame ended while idle (processes CTS, ICR, TF, etc.)
- `HandleFinishTX_StateRxData` — DATA reception complete (ACK generation)
- `HandleFinishTX_StateTxData` — concurrent transmission finished
- `HandleFinishTX_StateRxIcf`, `HandleFinishTX_StateRxIcr`, etc. for MAPC states.

6.2 Transmission Flow

Figure 2 shows the standard four-way handshake (RTS/CTS/DATA/ACK).

Apart from that, other transmission-exchange sequences are defined, thereby granting flexibility to the FSM. The list of sequences is provided in Table 3.

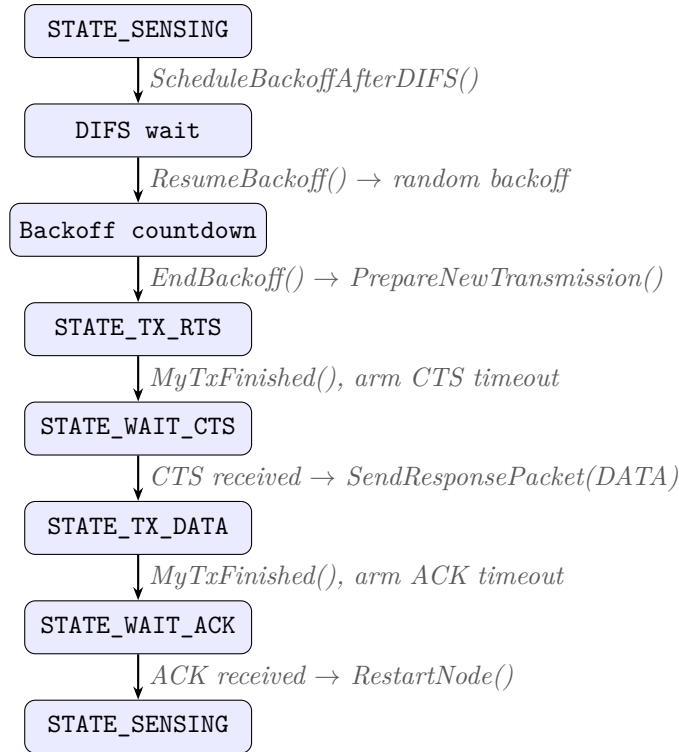


Figure 2: Standard RTS/CTS/DATA/ACK transmission flow.

Mode	Frames	Usage
No RTS/CTS	DATA → ACK	Direct short-range TX
RTS/CTS	RTS → CTS → DATA → ACK	Standard collision protection
Co-TDMA	ICF → ICR → [MU-RTS → DATA → ACK] × N	MAPC time-division
Co-BF/Co-SR	ICF → ICR → TF → DATA → ACK_TF → ACK	MAPC coordinated BF/SR
DSO	DSO_ICF → DSO_ICR → DATA → ACK	Dynamic subband
NPCA	NPCA_ICF → NPCA_ICR → DATA → ACK	Non-primary channel

Table 3: Transmission-exchange sequences.

6.3 Interruption Handling

When a node in `STATE_SENSING` receives `InportSomeNodeStartTX`:

- **Own frame** (RTS/CTS/DATA addressed to me): transition to appropriate RX state.
- **Foreign frame above CCA**: freeze backoff, set NAV, transition to `STATE_NAV`.
- **Foreign frame below CCA**: ignore (continue backoff).

6.4 Backoff Mechanism

The backoff subsystem is implemented across two files:

- `backoff_methods.h` — standalone functions: `ComputeBackoff()`, `PickNewCW()`, `ResetCW()`, `ComputeEDCABackoff()`.
- `node_backoff_methods.h` — node methods: `ResumeBackoff()`, `PauseBackoff()`, `EndBackoff()`, `RestartNode()`, `HandleSlottedBackoffCollision()`.

Supported backoff types:

Type	Value	Description
BACKOFF_DCF	0	Standard DCF exponential backoff
BACKOFF_EDCA	1	Per-AC BEB with AIFSN
BACKOFF_TOKENIZED	2	Token-based channel access
BACKOFF_DETERMINISTIC	3	Qualcomm deterministic
BACKOFF_REPEAT_BO	4	Reuse last successful backoff
BACKOFF_ECA	5	Enhanced channel access
BACKOFF_SYNCHRONIZED	6	Fixed slot allocation
BACKOFF_ML_ACCESS	7	ML-learned (external Python server)

`EndBackoff()` is the critical dispatch point: it calls `PrepareNewTransmission()` (for regular unicast), or initiates MAPC ICF sequences (for coordinator APs), selecting the active group via round-robin:

```
mapc_active_group_idx = mapc_coordinator_group_rr % wlan.
    num_mapc_groups;
mapc_coordinator_group_rr++;
```

6.5 Packet Handling

`node_packet_methods.h` contains the core transmission logic.

6.5.1 PrepareNewTransmission()

Called when a node is ready to transmit. This method:

1. Selects the channel bonding width via `GetTxChannels()`.
2. Computes A-MPDU aggregation level based on MCS, bandwidth, and TXOP budget.
3. Sets beamforming weights if applicable (see Section 9).
4. Calls `GenerateNotification()` to build the `Notification` struct.

6.5.2 GenerateNotification()

Populates a `Notification` with all `TxInfo` fields: source/destination IDs, packet type, channel range, TX power, NAV time, MCS, beamforming parameters, MAPC fields, and spatial-reuse state. If spatial reuse is active (`sr_state.mapc_cosr_active` or legacy SR), the reduced TX power from `sr_state.current_tx_power_sr` is used instead of the default.

6.5.3 StartTransmission()

Fires `outportSelfStartTX(notification)`, which broadcasts to all other nodes. A critical ordering constraint: `STATE_TX_TF` must be checked *before* the `exchange_sequence` check, because TF is sent mid-sequence while the first frame type is still ICF.

6.5.4 MyTxFinished()

Handles the `trigger_toFinishTX` event for every TX state. Broadcasts a finish notification via `outportSelfFinishTX()`, then transitions to the appropriate next state (e.g., `STATE_WAIT_ACK` after DATA, `STATE_SENSING` after ACK).

6.5.5 SendResponsePacket()

The SIFS timer handler. Generates and sends response frames: ACK (after DATA), CTS (after RTS), ICR (after ICF), or NPCA-ICR, depending on `node_state`.

6.5.6 Timeout Handlers

Defined in `node_timeout_methods.h`:

- `AckTimeout()` — No ACK received; Modify channel access parameters (e.g., increment CW), retry or drop the frame.
- `CtsTimeout()` — No CTS after RTS; modify channel access parameters (e.g., increment CW), retry.
- `DataTimeout()` — No DATA after CTS sent; return to sensing.
- `NpcaIcrTimeout()` — Safety fallback if no NPCA ICR is received.

7 Data Structures

7.1 NodeParameters

Per-node configuration loaded from the node CSV.

```
struct NodeParameters {
    int node_id;
    int node_type;           // NODE_TYPE_AP (0), NODE_TYPE_STA (1)
    char wlan_code[20];
    double x, y, z;          // Position (meters)
    int primary_channel;
    int min_channel_allowed, max_channel_allowed;
    int channel_bonding_model;
    double tx_power_default; // dBm
    double cca_default;      // dBm
    int traffic_model;
    int ieee_protocol;
};
```

Listing 6: NodeParameters (key fields)

7.2 Wlan

WLAN-level configuration grouping an AP and its associated STAs.

```
struct Wlan {
    int wlan_id;
    char wlan_code[20];
    int ap_id;
    int num_stas;
    int *list_sta_id;

    // MAPC (up to MAX_MAPC_GROUPS_PER_WLAN = 8)
    int mapc_enabled, num_mapc_groups;
    int mapc_group_ids[8], mapc_method_ids[8];
    int *mapc_peer_ap_ids[8];
    int FindMapcGroupIdx(int group_id);

    // Beamforming
    int bf_enabled, bf_N_elements;
    double bf_d_spacing, bf_az_deg;

    // DSO / NPCA / EDCA / SR ...
};
```

Listing 7: Wlan (key fields)

7.3 Notification and TxInfo

The message passed between nodes on every transmission event.

```
struct TxInfo {
    int packet_type, source_id, destination_id;
    int left_channel, right_channel;
    double tx_power, nav_time;
    double data_duration, ack_duration, rts_duration, cts_duration;
    int bits_ofdm_sym, modulation, num_packets_aggregated;

    // Beamforming
    int beamforming_active, beam_N_elements, beam_num_nulls,
        beam_use_zf;
    double beam_d_spacing, beam_az_main_rad;
    double beam_null_az_rad[MAX_BEAM_NULLS];

    // MAPC
    int mapc_group_id;
    double mapc_allocated_data_duration, mapc_sr_peer_tx_power;

    // Spatial Reuse
    int srg_obss_pd;
    double non_srg_obss_pd;

    // Preamble puncturing
    int punctured_channels[MAX_CHANNELS];
};

struct Notification {
    TxInfo tx_info;
    double timestamp;
};
```

Listing 8: TxInfo (key fields)

7.4 Performance

Reported from Node to Agent at the end of each measurement window.

```
struct Performance {
    double throughput, max_bound_throughput;
    double data_packets_sent, data_packets_lost;
    double rts_cts_sent, rts_cts_lost;
    double num_tx_init_not_possible;
    double average_delay, max_delay, min_delay;
    double average_utilization;
    double successful_channel_occupancy;
    double *rssi_list; // Per-WLAN RSSI (pW)
};
```

7.5 Configuration and Action

```
struct Configuration {
    double timestamp;
    int selected_primary_channel;
    double selected_pd; // pW
    double selected_tx_power; // pW
};
```

```

    int selected_max_bandwidth;    // num channels
    int spatial_reuse_enabled;
    double non_srg_obss_pd;        // pW
};

struct Action {
    int id, channel;
    double cca, tx_power;          // pW
    int max_bandwidth;
    double instantaneous_reward, cumulative_reward;
    int times_played;
};

```

8 Channel and Power Model

`power_channel_methods.h` implements the PHY-layer abstractions.

8.1 Path Loss and Received Power

`ComputePowerReceived()` computes the power received at a given distance using configurable path-loss models (free-space, log-distance, IEEE 802.11ax indoor, etc.). The received-power matrix is pre-computed at simulation start to save computation time, but it is updated upon specific events (e.g., changes in transmission power or position).

8.2 SINR Computation

`ComputeSINR()` sums interference from all concurrently active transmitters on overlapping channels:

$$\text{SINR} = \frac{P_{\text{signal}}}{N_0 + \sum_{i \in \mathcal{I}} P_{\text{interference}, i}}$$

8.3 Channel Bonding

`GetTxChannels()` implements multiple channel-bonding models:

Model	Value	Description
CB_ONLY_PRIMARY	0	Primary channel only (20 MHz)
CB_ALWAYS_MAX	2	Use all channels if all are free
CB_ALWAYS_MAX_LOG2	3	Widest contiguous power-of-2 bandwidth
CB_PREAMBLE_PUNCTURING	6	Allow punctured subchannels

`GetTxChannelsNPCA()` is the NPCA variant that checks secondary channels for non-primary channel access operation.

8.4 Power Sensing Update

`UpdatePowerSensedPerNode()` updates per-node power tracking when a new transmission starts. It takes receiver coordinates (`rx_x`, `rx_y`, `rx_z`) and multiplies `pw_received` by the beamforming gain—but *only for DATA frames*:

```

if (notification.packet_type == PACKET_TYPE_DATA) {
    double bf_gain = ComputeRxBeamGain(tx_info, tx_x, tx_y, rx_x, rx_y
    );
    pw_received *= bf_gain;
}

```

Control frames (ICF, TF, RTS, CTS, ACK) remain omnidirectional to prevent coordination deadlocks.

9 Beamforming

`beamforming_methods.h` implements horizontal Uniform Linear Array (ULA) beamforming with two null-placement algorithms.

9.1 Array Model

The phase shift for element n steering toward azimuth θ is:

$$\phi_n = \exp(j \cdot 2\pi \cdot d \cdot n \cdot \sin(\theta))$$

where d is the element spacing in wavelengths.

9.2 Algorithms

`ComputeBeamWeights()` — Projection Null-Steerer (Gram–Schmidt)

Used for standalone beamforming. Steers the main beam and places nulls via Gram–Schmidt orthogonalization. Set `beam_use_zf = 0`.

`ComputeZFBeamWeights()` — Zero-Forcing Precoding

Used for Co-BF (coordinated beamforming). Computes $\mathbf{w} = \mathbf{H}(\mathbf{H}^H \mathbf{H})^{-1} \mathbf{e}_1$, guaranteeing exact simultaneous nulls. Includes `SolveComplexLinearSystem()` (Gaussian elimination with partial pivoting, max 5×5). Set `beam_use_zf = 1`.

`ComputeRxBeamGain()` returns the beam gain at the receiver’s angle.

9.3 Integration with Transmission

In `PrepareNewTransmission()`:

- **Standalone BF**: projection null-steerer, `beam_use_zf = 0`.
- **Co-BF coordinator**: ZF with nulls toward peer STAs, `beam_use_zf = 1`.
- **Co-BF coordinated AP**: ZF with null toward coordinator’s STA.
- **Co-TDMA / Co-SR**: projection if BF enabled, `beam_use_zf = 0`.

Constraints: `BF_MAX_N_ELEMENTS = 32` (stack arrays). ZF fallback to projection if $N < K + 1$ or $K + 1 > \text{MAX_ZF_CONSTRAINTS}$ (5).

10 Backoff models

Different backoff types are implemented (see Table 6).

Type	Value	Description
BACKOFF_DCF	0	Standard DCF exponential backoff
BACKOFF_EDCA	1	Per-AC BEB with AIFSN

BACKOFF_TOKENIZED	2	Token-based channel access
BACKOFF_DETERMINISTIC	3	Qualcomm deterministic
BACKOFF_REPEAT_BO	4	Reuse last successful backoff
BACKOFF_ECA	5	Enhanced channel access
BACKOFF_SYNCHRONIZED	6	Fixed slot allocation

Table 6: Backoff types.

In `BACKOFF_DCF` and `BACKOFF_EDCA`, the contention window follows binary exponential back-off, whereby the contention window (CW) is updated upon experiencing packet losses:

$$CW_{\text{new}} = \min(2 \cdot CW_{\text{current}} + 1, CW_{\text{max}})$$

On successful transmission, CW resets to `CW_min`.

EDCA differentiates traffic into four Access Categories (AC):

AC	Traffic	Typical CW
AC_VO (0)	Voice	[8, 16]
AC_VI (1)	Video	[16, 32]
AC_BE (2)	Best Effort	[32, 1024]
AC_BK (3)	Background	[32, 1024]

Each AC has its own contention window bounds (`ac_cw_min[]`, `ac_cw_max[]`), arbitration inter-frame spacing (`ac_aifsn[]`), and TXOP limit (`ac_txop_limit[]`). `ComputeEDCABackoff()` in `backoff_methods.h` applies the per-AC AIFSN and CW parameters.

11 Multi-AP Coordination (MAPC)

MAPC enables coordinated operation between multiple BSSs.⁷ Three methods are supported:

Method	Constant	Description
Co-TDMA	MAPC_CO_TDMA (1)	Time-division among coordinated APs
Co-SR	MAPC_CO_SR (2)	Coordinated spatial reuse with power adaptation
Co-BF	MAPC_CO_BF (3)	Coordinated beamforming with ZF nulls

11.1 Frame Exchange Sequence

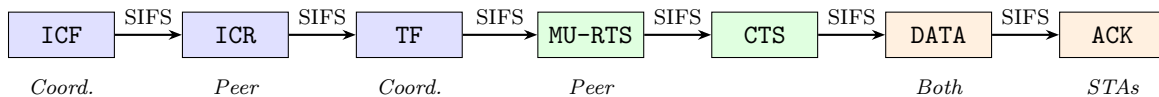


Figure 3: MAPC Co-TDMA frame exchange.

1. **ICF** (Initial Control Frame): Coordinator broadcasts to its MAPC group (`destination_id = NODE_ID_MAPC_BROADCAST`).

⁷Wilhelmi, F., Bellalta, B., Geraci, G., Galati-Giordano, L., Meneghello, F., Kijanka, A., Val, I., & López-Pérez, D. (2026). A Tutorial on IEEE 802.11 bn Multi-AP Coordination for Wi-Fi 8: From Standardization to Performance Evaluation. arXiv preprint arXiv:2606.13759.

2. **ICR** (ICF Response): Coordinated APs acknowledge readiness.
3. **TF** (Trigger Frame): Coordinator allocates time slots.
4. **MU-RTS**: Coordinated AP solicits CTS from its STA.
5. **CTS / DATA / ACK**: Per-AP data exchange within the allocated TXOP.

11.2 Multi-Group Support

Each WLAN can participate in up to `MAX_MAPC_GROUPS_PER_WLAN` (8) groups. The `Wlan` struct stores per-group arrays: `mapc_group_ids[]`, `mapc_method_ids[]`, `mapc_peer_ap_ids[]`, etc. The coordinator round-robins through groups via `mapc_coordinator_group_rr`, but other implementations are possible.

11.3 Co-TDMA TXOP Split

In Co-TDMA, the coordinator AP divides the TXOP equally among active APs as a baseline implementation. In `ProceedAfterIcr()`: $\text{per_ap_duration} = (\text{TXOP}_{\text{max}} - \text{overhead}) / N_{\text{active}}$, embedded in `mapc_allocated_data_duration` of the TF notification.

11.4 Co-SR Power Computation

In the baseline implementation of Co-SR, `ComputeCoSRTxPowers()` returns the pre-configured power from `wlan.mapc_sr_tx_power_dbm[group_idx]`, which is indicated by the user through the MAPC input file. The coordinator sets `sr_state.mapc_cosr_active = TRUE` and embeds the peer's allocated power in `tf_notification.tx_info.mapc_sr_peer_tx_power`.

12 NPCA and DSO

12.1 DSO (Dynamic Subband Operation)

DSO allows a BSS to use a secondary subband after winning contention on the primary. The flow is:

1. AP sends DSO_ICF (non-HT duplicate on full BSS bandwidth).
2. Timer-modeled ICR response on secondary subband.
3. DATA transmission on the DSO secondary subband.
4. Primary channel is freed during DATA.

Timing overhead: $\sim 136 \mu\text{s}$ (ICF + switch + ICR + SIFS).

12.2 NPCA (Non-Primary Channel Access)

NPCA allows a BSS to access secondary channels without contending on the primary. The operation works as follows:

1. AP detects inter-BSS PPDU on primary, freezes primary backoff.
2. AP switches to the NPCA primary channel, which takes a switch time delay (set to $16 \mu\text{s}$ by default).
3. AP sends NPCA ICF on the NPCA primary channel (`STATE_TX_NPCA_ICF`).
4. STA detects ICF finish in `HandleFinishTX_StateSensing` (guard: `npca_enabled && npca_primary_channel >= 0 && node_type != AP`).
5. STA sends ICR after SIFS (`STATE_TX_NPCA_ICR`).
6. AP receives ICR in `InportSomeNodeFinishTX` case `STATE_RX_NPCA_ICR`.
7. AP cancels safety timeout and begins DATA transmission (`STATE_TX_DATA_NPCA`).
8. AP returns to the primary channel, which again takes a switch time delay.

A safety timeout (SIFS + switch time + ICR duration $\approx 52 \mu\text{s}$) is kept as a fallback but fires 0 times in validated scenarios.

13 Machine Learning Subsystem

The ML subsystem enables online parameter tuning through a modular pipeline. A node (AP) can have an associated agent running a certain ML algorithm (e.g., MABs with ϵ -greedy) to do a certain operation (e.g., joint channel and transmission power optimization). Agents can be associated with a central controller (optional), which may centralize the optimization operation.



Figure 4: ML subsystem architecture.

13.1 Agent's Flow

In `Agent::ComputeNewConfiguration()`:

1. `arm_ix = pre_processor.EncodeConfiguration(config, from_ap)` — current config $\rightarrow$ action index.
2. `reward = pre_processor.ComputeReward(type, performance)` — performance $\rightarrow$ scalar reward.
3. `ml_output = learning_algorithm.Update(arm_ix, reward, available_arms)` — get new action.
4. `pre_processor.DecodeAction(ml_output, &new_config)` — action $\rightarrow$ new configuration.
5. `outportSendToNode(new_config)` — apply to node.

13.2 Learning Algorithms

Multi-Armed Bandits (`multi_armed_bandits.h`)

Three strategies: ϵ -greedy, UCB1 (Upper Confidence Bound), and Thompson Sampling (Beta distributions). Action space is the Cartesian product of channels $\times$ CCA values $\times$ power levels $\times$ sensitivity levels.

RTOT (`rtot_algorithm.h`)

Recommended TX Output Threshold. Adjusts `non_srg_obss_pd` with a step-size, bounded by `[min, max]`. Direction reverses when reward decreases. Note: `ml_output` is `double` (not `int`) to handle RTOT's continuous output correctly.

External Model (`external_model_client.h`)

Socket communication with an external Python ML server. C++ sends state vector (`uint32_t n_feat`, then `n_feat  $\times$  float32`), receives action vector. Enables DRL and other advanced algorithms.

13.3 Reward Types

Value	Constant	Formula
1	<code>REWARD_TYPE_PACKETS_SUCCESSFUL</code>	$(\text{sent} - \text{lost}) / \text{sent}$
2	<code>REWARD_TYPE_AVERAGE_THROUGHPUT</code>	$\text{throughput} / \text{max_bound}$
3	<code>REWARD_TYPE_MIN_RSSI</code>	$\text{min}(\text{rssi_per_sta})$
4	<code>REWARD_TYPE_MAX_DELAY</code>	$1.0 / \text{max_delay}$
5	<code>REWARD_TYPE_MIN_DELAY</code>	$1.0 / \text{min_delay}$
6	<code>REWARD_TYPE_AVERAGE_DELAY</code>	$1.0 / \text{average_delay}$

13.4 External ML Server Integration

Python servers in `Code/learning_modules/python_servers/`:

- `ml_server_passthrough.py` — Smoke-test: echoes arm index unchanged.
- `ml_server_pytorch.py` — PyTorch JIT inference with DQN training hook.
- `ml_server_random.py` — Baseline random action selection.

Wire protocol: C++ sends `[uint32 n_feat] [n_feat × float32]`, Python returns `[float32 action]`.

13.5 Central Controller

`CentralController` (`central_controller.h`) coordinates multi-WLAN learning. It receives configurations from all agents, can ban/allow actions via `CentralizedActionBanning`, and distributes coordinated commands.

14 Input Configuration

All scenarios are defined through semicolon-delimited CSV files.

14.1 Node CSV

Each row defines one node (AP or STA). Key columns:

Col	Field	Description
0	<code>node_code</code>	Identifier (AP_A, STA_B1, etc.)
1	<code>node_type</code>	0 = AP, 1 = STA
2	<code>wlan_code</code>	WLAN identifier (string, e.g. “A”)
3–5	<code>x, y, z</code>	Position in meters
6	<code>central_freq</code>	Center frequency (GHz)
7	<code>cb_model</code>	Channel bonding model
8	<code>primary_channel</code>	Primary channel index
9–10	<code>min/max_channel</code>	Allowed channel range
11	<code>tx_power</code>	TX power (dBm)
12	<code>sensitivity</code>	Receiver sensitivity (dBm)
13	<code>traffic_model</code>	Traffic generation model
14	<code>traffic_load</code>	Packet generation rate
...	...	(backoff, CW, RTS/CTS, BSS color, SR, ...)
28–31	<code>bf_enabled, bf_N, bf_d, bf_az_deg</code>	Beamforming config (optional)
37	<code>dso_enabled</code>	DSO enable (0/1)
38	<code>npca_enabled</code>	NPCA enable (0/1)
39	<code>npca_primary_ch</code>	NPCA primary channel

14.2 MAPC CSV

```
GroupID;Method;CoordinatedBSSIDs;ExtraParams
1;CO_TDMA;A,B;
2;CO_SR;B,C;20.0
```

Listing 9: MAPC configuration format

- **GroupID:** Integer group identifier.
- **Method:** CO_TDMA, CO_SR, or CO_BF.
- **CoordinatedBSSIds:** Comma-separated WLAN codes (string matching).
- **ExtraParams:** Method-specific (e.g., TX power in dBm for Co-SR).

14.3 Agent CSV

Each row defines one ML agent. Key columns: `wlan_code`, `centralized` (0/1), `time_between_requests` (seconds), `actions channels`, `actions cca` (dBm, comma-separated), `actions tx power` (dBm, comma-separated), `max_bandwidth`, `reward` (type constant), `learning mechanism`, `selected_strategy`.

14.4 Example Run

```
cd Code/main
./komondor_main \
    --nodes ../input/examples/basic_examples/input_nodes.csv \
    --time 10.0 \
    --seed 1

# With agents:
./komondor_main \
    --nodes ../input/examples/basic_examples/input_nodes.csv \
    --agents ../input/examples/basic_examples/agents.csv \
    --time 10.0 --seed 1

# With MAPC:
./komondor_main \
    --nodes ../input/examples/mapc_example/input_nodes_mapc_example.csv \
    --mapc ../input/examples/mapc_example/mapc_cotdma_example.csv \
    --time 10.0 --seed 1
```

Listing 10: Running a simulation

15 Output

Simulation results are written to `Code/output/` and include per-node statistics:

- Throughput (Mbps)
- Packets sent / lost / ACKed
- Average, min, max delay
- Channel utilization and occupancy
- RSSI and SINR measurements
- Spatial reuse and MAPC performance metrics

The output formatting is handled by `output_methods.h`, invoked from `komondor_main.cc` after the simulation completes. Per-node log files are written if `save_node_logs` is enabled, using the `LOGS()` macro:

```
LOGS(save_node_logs, node_logger.file, "%.15f;N%d;%s;%s\n",
     SimTime(), node_params.node_id, "StartTX", "DATA");
```

16 Coding Conventions

- **C++98 strict:** No C++11 features. No `auto`, no lambdas, no in-class member initializers, no range-based `for`, no `nullptr`. Use `NULL`, constructor initializer lists, and `explicit`

type declarations.

- **Tabs** for indentation (not spaces).
- **Method signatures:** `void Node :: MethodName(...)` with spaces around `::`.
- **LOGS macro:** `LOGS(save_node_logs, node_logger.file, ...)` — always preserve when editing.
- **Header-only:** almost all logic in `.h` files. The COST preprocessor requires this for component methods.
- **Constants:** all-caps macros in `list_of_macros.h` (e.g., `STATE_SENSING`, `PACKET_TYPE_DATA`).
- **loss_reason:** a member variable (not local).
- **nav_collision / inter_bss_nav_collision:** local variables declared inside each handler.

17 Adding a New Feature: Checklist

When adding a new feature to Komondor, follow this checklist:

1. **Define constants** in `list_of_macros.h` (new states, packet types, default values).
2. **Extend structures** in `Code/structures/` if new per-node, per-WLAN, or per-notification fields are needed.
3. **Add CSV columns** and parsing logic in `input_loader.h`. Use `GetField()` with null-checks for backward compatibility.
4. **Implement logic** in the appropriate `Code/methods/` header file.
5. **Wire into the FSM** by adding cases to the `HandleStartTX_*` / `HandleFinishTX_*` dispatch in `node.h`.
6. **Handle timer events** by adding new timers in `node.h` and their handlers in `node_timeout_methods.h` or `node_backoff_methods.h`.
7. **Initialize** new member variables in `InitializeVariables()` and reset them in `RestartNode()`.
8. **Create test scenarios** in `Code/input/validation/`.
9. **Structural verification:** Use a brace-counting script to verify `node.h` integrity after any switch-case modifications (expected depth = 4 at EOF due to outer component block).

18 Common Pitfalls

Lessons learned from the refactoring history:

1. **Function extraction:** When extracting a code block into a helper, the original body *must* be removed from the call site. Leaving both causes double execution (double buffer fill, double trigger firings, double stats counting).
2. **Broadcast frame destination:** ICF and TF use `NODE_ID_MAPC_BROADCAST (-2)`, not a specific node ID. Finish notifications must preserve this value.
3. **BF on control frames:** Beamforming gain must be DATA-only. Applying BF to control frames (ICF, RTS, CTS, ACK) causes coordination deadlocks because ULA nulls can zero out control-frame power.
4. **Integer truncation:** The RTOT algorithm returns a `double` (power in pW). Casting to `int` gives 0. Use `double` throughout the ML pipeline.

5. **atoi() on string codes:** `atoi("A") = atoi("B") = 0`. Use `std::string` comparison for WLAN codes.
6. **NAV time in ICF handler:** The coordinated AP's ICF handler copies duration fields but historically missed `nav_time`. Third-party APs crash if `nav_time = 0`.
7. **StartTX TF ordering:** `STATE_TX_TF` must be checked before the `exchange_sequence` check in `StartTransmission()`.
8. **Python patch scripts:** A `sys.exit(1)` on any failed patch aborts before the file write, losing all in-memory patches. Split into per-file scripts.
9. **C++98 constraints:** No `auto`, no lambdas, no in-class initializers, no range-based for loops. Use `NULL`, constructor initializer lists, and explicit type declarations.

19 Quick Reference: Key Methods

Table 11: Key methods quick reference

Method	File	Purpose
<code>Setup()</code>	fsm	One-time init at simulation start
<code>Start()</code>	fsm	Begin traffic gen & first backoff
<code>InitializeVariables()</code>	fsm	Reset all state to defaults
<code>RestartNode()</code>	backoff	Soft reset after TXOP
<code>ScheduleBackoffAfterDIFS()</code>	backoff	Arm DIFS → backoff chain
<code>ResumeBackoff()</code>	backoff	Resume after DIFS/AIFS expiry
<code>PauseBackoff()</code>	backoff	Freeze backoff on busy channel
<code>EndBackoff()</code>	backoff	Dispatch: regular TX or MAPC ICF
<code>PrepareNewTransmission()</code>	packet	Aggregation, CB, BF, build notification
<code>GenerateNotification()</code>	packet	Populate TxInfo fields
<code>StartTransmission()</code>	packet	Fire <code>outportSelfStartTX</code>
<code>MyTxFinished()</code>	packet	Handle <code>trigger_toFinishTX</code>
<code>SendResponsePacket()</code>	packet	SIFS handler: send ACK/CTS/ICR
<code>ProceedAfterIcr()</code>	packet	MAPC: decide next step after ICR
<code>ComputeCoSRTxPowers()</code>	packet	Co-SR power computation
<code>AckTimeout()</code>	timeout	No ACK → retry or drop
<code>CtsTimeout()</code>	timeout	No CTS → retry
<code>DataTimeout()</code>	timeout	No DATA → sensing
<code>NavTimeout()</code>	timeout	NAV expired → new backoff
<code>ComputeBackoff()</code>	mac/bo	Random backoff in $[0, CW]$
<code>ComputeRxBeamGain()</code>	bf	Beam gain at receiver angle
<code>ComputeBeamWeights()</code>	bf	Projection null-steerer
<code>ComputeZFBeamWeights()</code>	bf	ZF precoding ($\mathbf{H}^{-1}$ approach)
<code>GetTxChannels()</code>	channel	Channel bonding selection
<code>UpdatePowerSensedPerNode()</code>	channel	Per-node power with BF gain
<code>ComputeSINR()</code>	channel	Signal-to-interference ratio

20 Glossary

BSS	Basic Service Set.
CB	Channel Bonding.
CCA	Clear Channel Assessment.
Co-BF	Coordinated Beamforming.
Co-SR	Coordinated Spatial Reuse.
Co-TDMA	Coordinated TDMA.
CompC++	Component-oriented C++.
CW	Contention Window.
DSO	Dynamic Spectrum Operation.
EDCA	Enhanced Distributed Channel Access.
EIFS	Extended Inter-Frame Spacing.
ICF	Initial Control Frame.
ICR	Initial Control Frame Response.
MAPC	Multi-AP Coordination.
MCS	Modulation and Coding Scheme.
MU-RTS	Multi-User RTS.
NAV	Network Allocation Vector.
NPCA	Non-Primary Channel Access.
OFDMA	Orthogonal Frequency-Division Multiple Access.
RTOT	Recommended TX Output Threshold.
SIFS	Short Inter-Frame Spacing.
SINR	Signal-to-Interference-plus-Noise Ratio.
SR	Spatial Reuse.
TF	Trigger Frame.
TXOP	Transmission Opportunity.
TXS	Transmission Status frame.
ULA	Uniform Linear Array.
ZF	Zero-Forcing.